

Lezione 2 - Unsupervised Tasks

- Clustering
- PCA
- Anomaly Detections

Clustering

Il clustering è un metodo di analisi dati utilizzato nell'apprendimento automatico e nella statistica per raggruppare dati simili in gruppi in cui gli oggetti all'interno dello stesso cluster risultano più simili tra di loro e meno simili rispetto a quelli appartenenti a cluster diversi.

Analizzando le relazioni o le similitudini tra i dati all'interno dello stesso cluster, inoltre, si possono scoprire informazioni non immediatamente evidenti quando si osserva l'insieme completo dei dati.

Esempi applicativi dove è possibile scoprire strutture dati ad esempio sono:

- Customer Segmentation.
- Ricerca di Mercato.
- Analisi delle recensioni online.
- Raggruppare documenti o immagini direttamente dal loro contenuto.

Algoritmi di Clustering

Ogni approccio di clustering può funzionare meglio per una particolare distribuzione di dati.

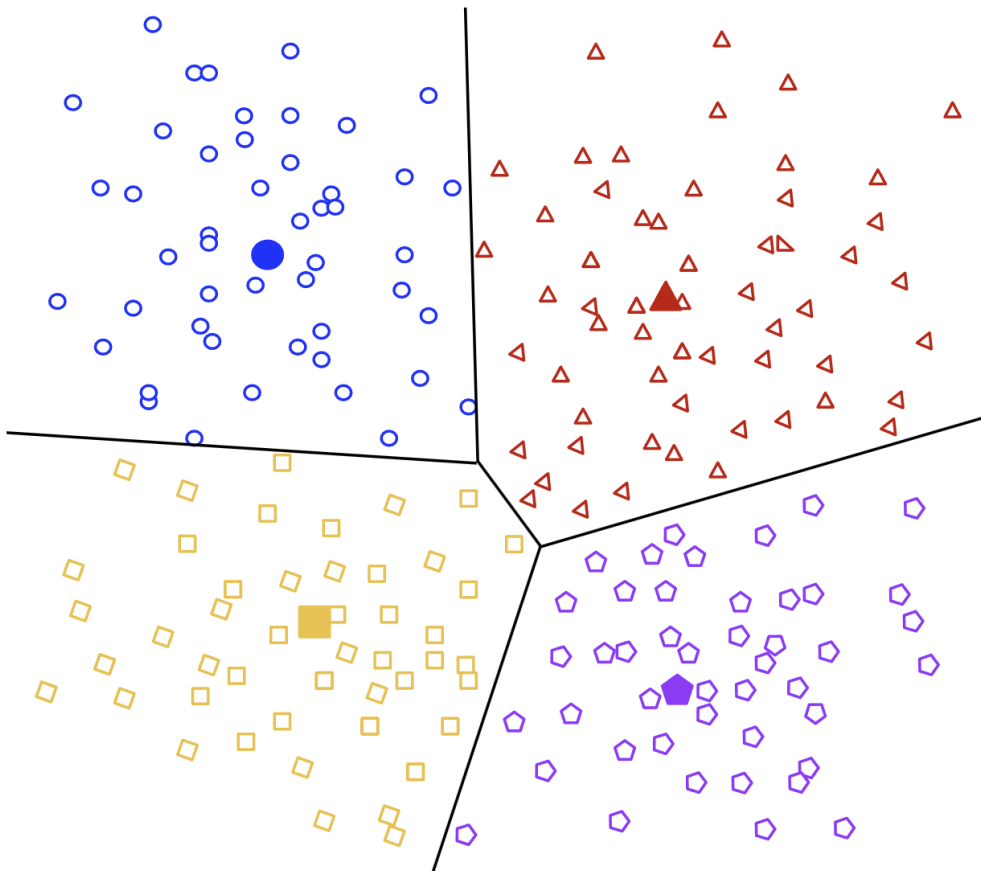
Vedremo 4 "famiglie" di algoritmi. In questo lavoro è presente una lista esaustiva:

<https://link.springer.com/article/10.1007/s40745-015-0040-1>

Centroid-based Clustering

Suddivide i dati in cluster in modo che ciascun cluster sia rappresentato da un "centroide," che è il punto medio o il centro geometrico di tutti gli oggetti nel cluster. Gli oggetti vengono assegnati

al cluster con il centroide più vicino. **KMeans** è l'esempio tipico per questo tipo di algoritmo.



Come applicare l'algoritmo di clustering di KMeans?

Facciamo ora un esempio per capire come funziona effettivamente K-Means:

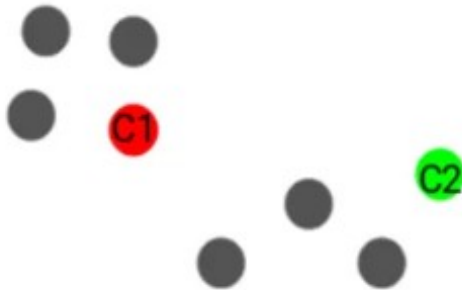


Abbiamo questi 8 punti e vogliamo applicare k-means per creare cluster per questi punti. Ecco come possiamo farlo.

Scegli il numero di cluster k Il primo passo in k-means è scegliere il numero di cluster, k.

1. Seleziona k punti casuali dai dati come centroidi

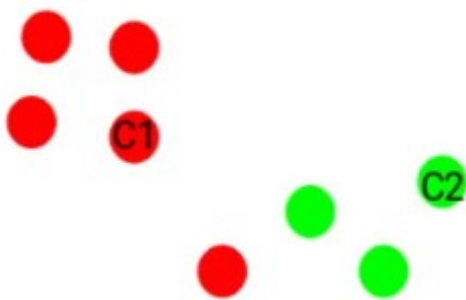
Successivamente, selezioniamo casualmente il centroide per ciascun cluster. Supponiamo di voler avere 2 cluster, quindi k è uguale a 2. Quindi selezioniamo casualmente il centroide:



Qui, i cerchi rossi e verdi rappresentano il centroide di questi cluster.

1. Assegna tutti i punti al centroide del cluster più vicino

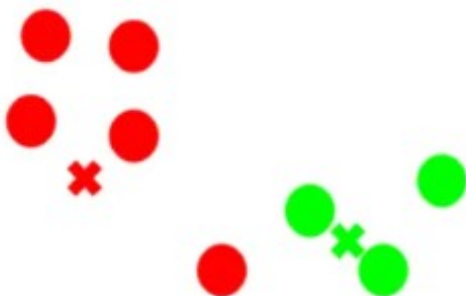
Una volta inizializzati i centroidi, assegniamo ogni punto al centroide del cluster più vicino:



Qui puoi vedere che i punti più vicini al punto rosso sono assegnati al cluster rosso, mentre i punti più vicini al punto verde sono assegnati al cluster verde.

1. Ricalcolare i centroidi dei cluster appena formati

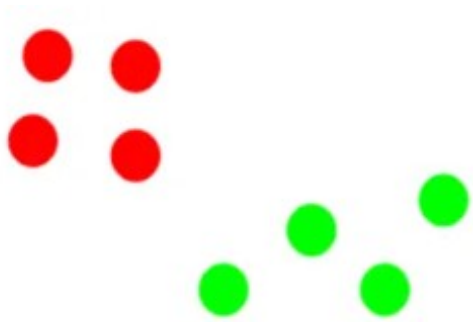
Ora, una volta assegnati tutti i punti a uno dei due cluster, il passo successivo è calcolare i centroidi dei cluster appena formati:



Qui le croci rossa e verde sono i nuovi centroidi.

1. Ripetere i passaggi 3 e 4

Ripetiamo quindi i passaggi 3 e 4:



Il passaggio di calcolo del centroide e di assegnazione di tutti i punti al cluster in base alla loro distanza dal centroide è un'unica iterazione . Ma aspetta: quando dovremmo interrompere questo processo? Non può durare per sempre, giusto?

Criteri di arresto per il clustering K-Means Esistono essenzialmente tre criteri di arresto che possono essere adottati per arrestare l'algoritmo K-means:

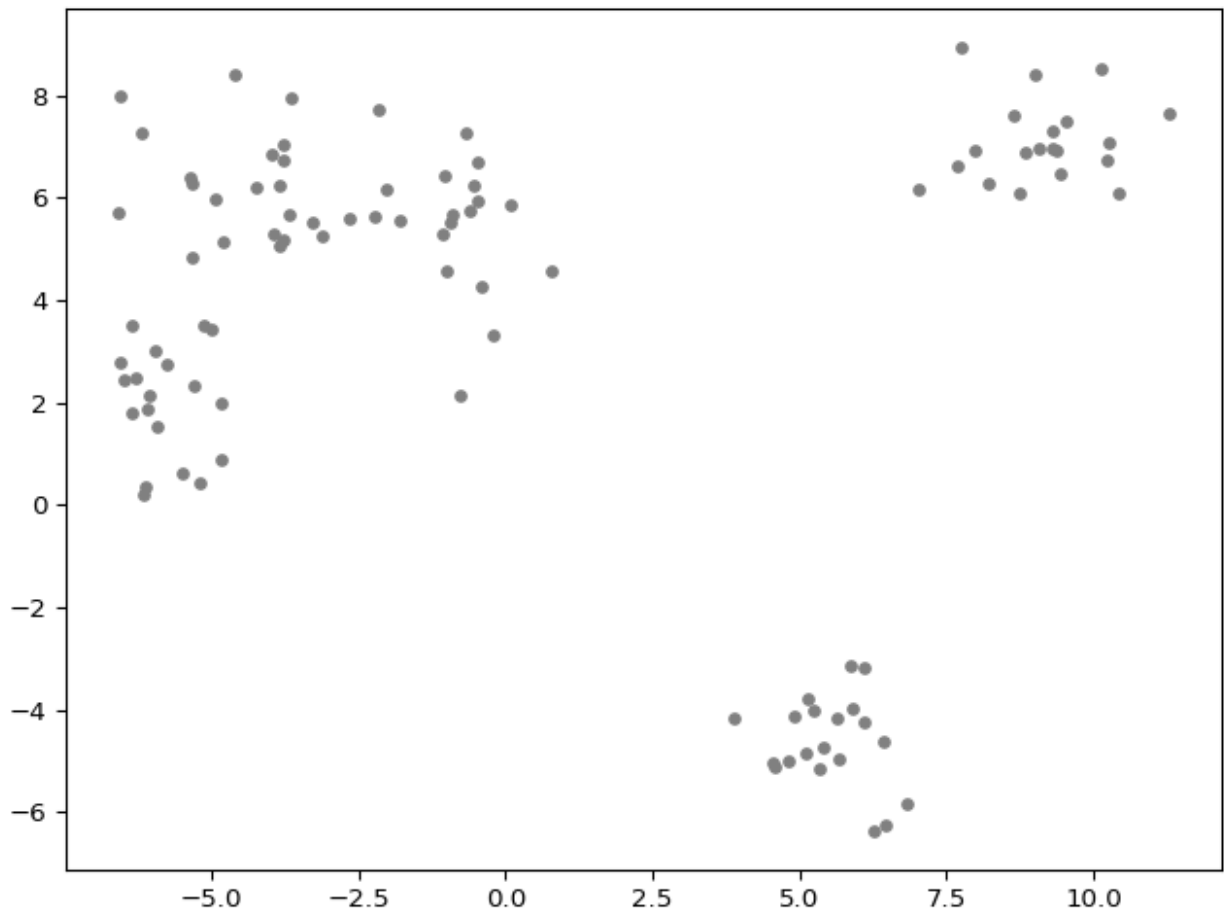
- I centroidi degli ammassi appena formati non cambiano
- I punti rimangono nello stesso cluster
- È stato raggiunto il numero massimo di iterazioni

Kmeans Esempio

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import random
# Generazione di un dataset casuale
from sklearn.datasets import make_blobs

data, _ = make_blobs(n_samples=100, centers=5, random_state=1234)
#samples numero totale di punti generati e centers numero di
raggruppamenti

plt.figure(num=0, dpi=96, figsize=(8, 6))
plt.scatter(data[:,0], data[:,1], c='grey', marker='o', s=16)
plt.show()
```



Di seguito un esempio di come utilizzare l'algoritmo K-means di Scikit-learn sul dataset appena generato.

Definiamo le nostre funzioni di utilities

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import matplotlib.patches as mpatches
from skimage import color, transform, feature
from sklearn.metrics import confusion_matrix
import itertools
import warnings
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

def separate_pattern_classes(data, labels, class_count):
    patternCount=data.shape[0]
    result = np.zeros(class_count, dtype = np.ndarray)
    for i in range(class_count):
```

```

        result[i] = np.array([data[j] for j in range(patternCount) if
labels[j]==i])

    return result

def compute_square_euclidean_distance(v1, v2):
    return sum(np.power(v1 - v2, 2))

#genera una griglia sul piano 2-D, colora lo sfondo in base al cluster
più vicino
# e visualizza punti e centroidi (X nera) per capire come K-Means
separa lo spazio.
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

def draw_kmeans_decision_boundaries(
    X,                # ndarray shape (n_samples, 2)
    labels,           # ndarray shape (n_samples,)
    centroids,        # ndarray shape (n_clusters, 2)
    step=0.05,        # passo della griglia
    figsize=(8, 6),   # dimensione figura
    borders=None,     # margine extra (dx, dy); se None lo calcola
al volo
    alpha_bg=0.30,    # trasparenza delle regioni di decisione
    cmap_name='tab10' # colormap matplotlib da cui attingere i
colori
):
    """
    Disegna le regioni di decisione del K-Means (Diagramma di
    Voronoi),
    i punti del dataset e i centroidi, usando solo librerie standard.
    """

    # -----
    # 1) Calcolo limiti + margini
    # -----
    # Estensione minima e massima dei dati
    x_min, x_max = X[:, 0].min(), X[:, 0].max()
    y_min, y_max = X[:, 1].min(), X[:, 1].max()

    # Calcolo dei margini
    if borders is None:
        dx = (x_max - x_min) * 0.05
        dy = (y_max - y_min) * 0.05
    else:
        dx, dy = borders

    # Applicazione dei limiti finali
    x_min, x_max = x_min - dx, x_max + dx

```

```

y_min, y_max = y_min - dy, y_max + dy

# -----
# 2) Mesh-grid (Creazione della griglia di punti di test)
# -----
xx, yy = np.meshgrid(np.arange(x_min, x_max, step),
                     np.arange(y_min, y_max, step))

mesh_points = np.c_[xx.ravel(), yy.ravel()] # Unione delle
coordinate (N_grid_points, 2)

# -----
# 3) Assegnazione a ogni punto della griglia
# -----
# Calcolo della distanza Euclidea:
# mesh_points[:, None, :] espande la dimensione (N_grid, 1, 2)
# centroids[None, :, :] espande la dimensione (1, N_clusters, 2)
# La sottrazione calcola la differenza per ogni coppia
dists = np.linalg.norm(mesh_points[:, None, :] -
centroids[None, :, :], axis=2) # Distanze (N_grid, N_clusters)

# Assegnazione del cluster (indice del minimo) e rimodellamento
alla forma della griglia
Z = dists.argmin(axis=1).reshape(xx.shape) # Z shape: (N_y,
N_x)

# -----
# 4) Impostazione colori
# -----
n_clusters = centroids.shape[0]
# Colormap di base (per i punti)
base_cmap = plt.cm.get_cmap(cmap_name, n_clusters)

# Colormap sfumata (per lo sfondo)
light_cmap = ListedColormap(base_cmap(np.arange(n_clusters))),
name='cluster_cmap')

# -----
# 5) Plotting
# -----
plt.figure(figsize=figsize, dpi=96)

# (a) Regioni di decisione (sfondo)
plt.pcolormesh(xx, yy, Z,
               cmap=light_cmap,
               shading='auto',
               alpha=alpha_bg)

# (b) Punti del dataset
plt.scatter(X[:, 0], X[:, 1],

```

```

        c=labels, cmap=base_cmap,
        edgecolor='k', s=40)

# (c) Centroidi
plt.scatter(centroids[:, 0], centroids[:, 1],
            marker='x', c='black',
            s=200, linewidths=2, label='Centroidi')

# -----
# 6. Finalizzazione estetica
# -----
plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)
# Rende gli assi uguali (importante per le distanze euclidee)
plt.gca().set_aspect('equal', adjustable='box')
plt.title('K-Means – Decision boundaries & centroids')
plt.legend()
plt.tight_layout()
plt.show()

# Esempio di Utilizzo (Per testare che la funzione sia autonoma)
if __name__ == '__main__':
    # Dati di esempio (Generazione di 3 cluster)
    np.random.seed(42)
    X = np.concatenate([
        np.random.normal(loc=[2, 2], scale=0.8, size=(50, 2)),
        np.random.normal(loc=[6, 7], scale=0.8, size=(50, 2)),
        np.random.normal(loc=[9, 2], scale=0.8, size=(50, 2))
    ])

    # Simulazione dei risultati
    centroids_final = np.array([[2.1, 2.1], [6.1, 7.1], [8.9, 1.9]])
    labels_final = np.array([0]*50 + [1]*50 + [2]*50)

```

Kmeans con Scikit Learn

La [libreria](#) Scikit-learn fornisce un'implementazione dei più importanti algoritmi di clustering, vedi [qui](#).

Di seguito un esempio di come utilizzare l'algoritmo K-means di Scikit-learn sul dataset appena generato.

```

import matplotlib.pyplot as plt                # necessario per la
funzione
from sklearn.cluster import KMeans

```



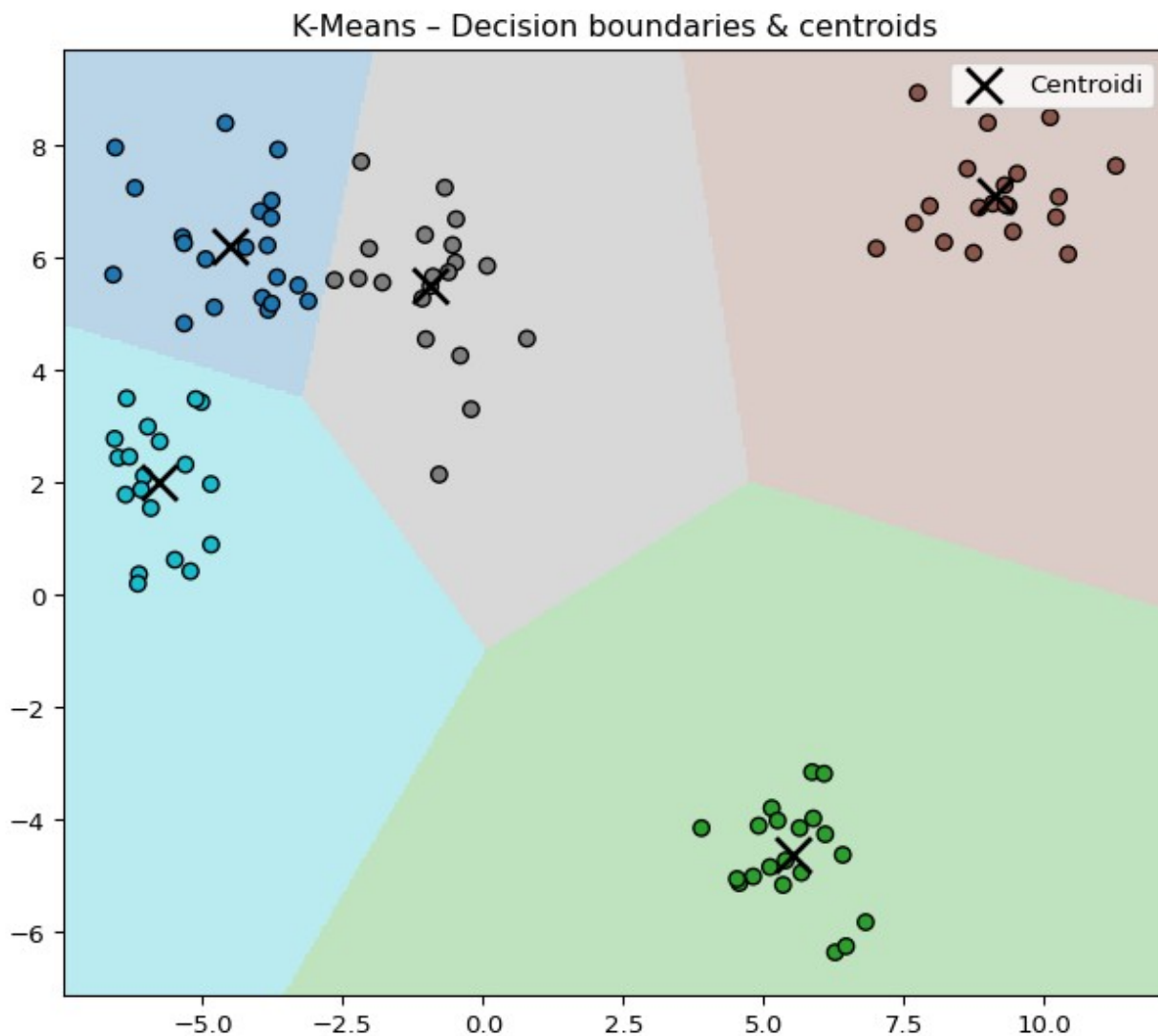
```

# 1. addestramento
n_clusters = 5
kmeans = KMeans(n_clusters=n_clusters, random_state=1234)
kmeans.fit(data)                                # data deve avere solo 2
colonne!

# 2. visualizzazione
draw_kmeans_decision_boundaries(
    X          = data,                          # punti 2-D
    labels     = kmeans.labels_,                # etichette assegnate
    centroids  = kmeans.cluster_centers_,       # centroidi
    step       = 0.05                           # risoluzione della griglia
)

/tmp/ipython-input-446623126.py:89: MatplotlibDeprecationWarning: The
get_cmap function was deprecated in Matplotlib 3.7 and will be removed
in 3.11. Use ``matplotlib.colormaps[name]`` or
``matplotlib.colormaps.get_cmap()`` or ``pyplot.get_cmap()`` instead.
base_cmap = plt.cm.get_cmap(cmap_name, n_clusters)

```



È possibile ottenere i centroidi dei vari cluster utilizzando l'attributo `cluster_centers_`:

```
print('Centroidi ottenuti:\n', kmeans.cluster_centers_)
```

```
Centroidi ottenuti:  
[[-4.50296061  6.22703212]  
 [ 5.51366983 -4.64045731]  
 [ 9.11338955  7.10348958]  
 [-0.95820208  5.50422472]  
 [-5.7920073   1.9979973  ]]
```

Come per i classificatori, in Scikit-learn gli algoritmi di clustering mettono a disposizione il metodo `.predict(...)` che permette di ottenere l'indice del cluster a cui un pattern viene assegnato.

```
# Esempio di uso della funzione predict
print('Cluster predetti:\n', kmeans.predict([[-2.5, 0.0], [2.5, -4.0],
[0.0, 5.0]]))
```

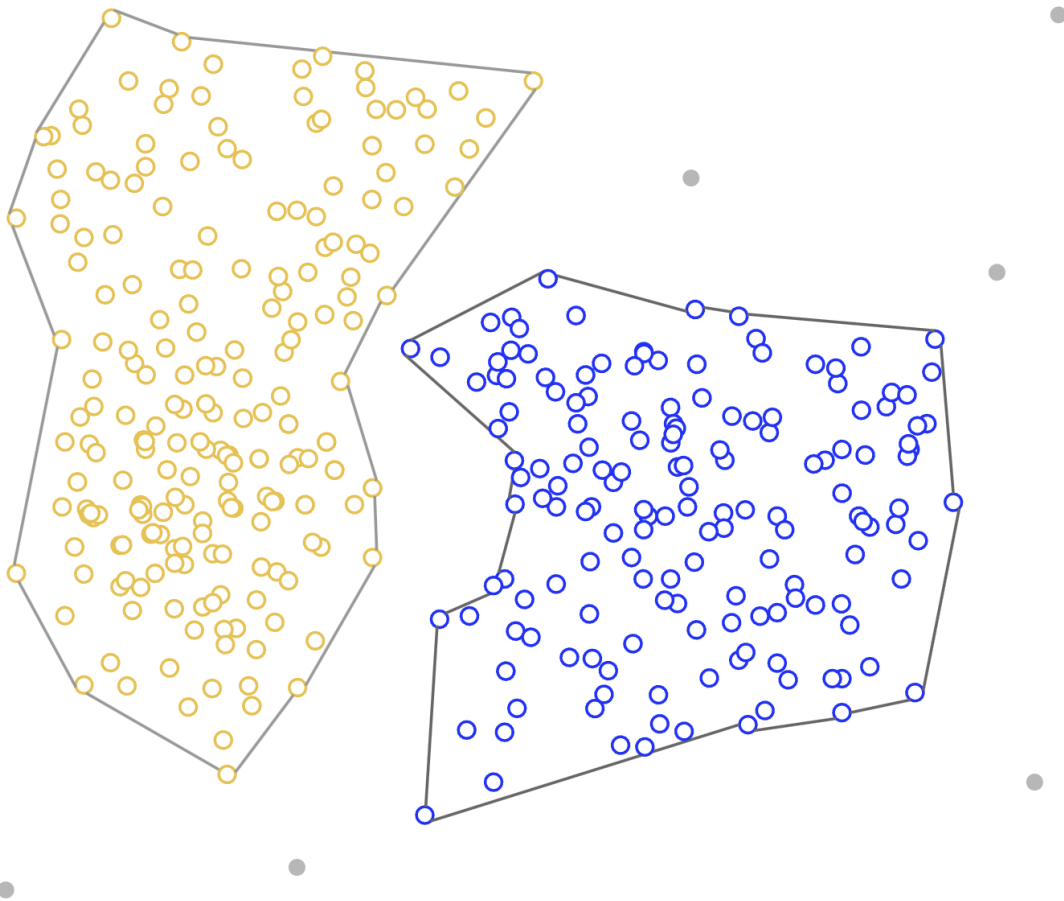
```
Cluster predetti:
[4 1 3]
```

Gli altri algoritmi di clustering

Density-based

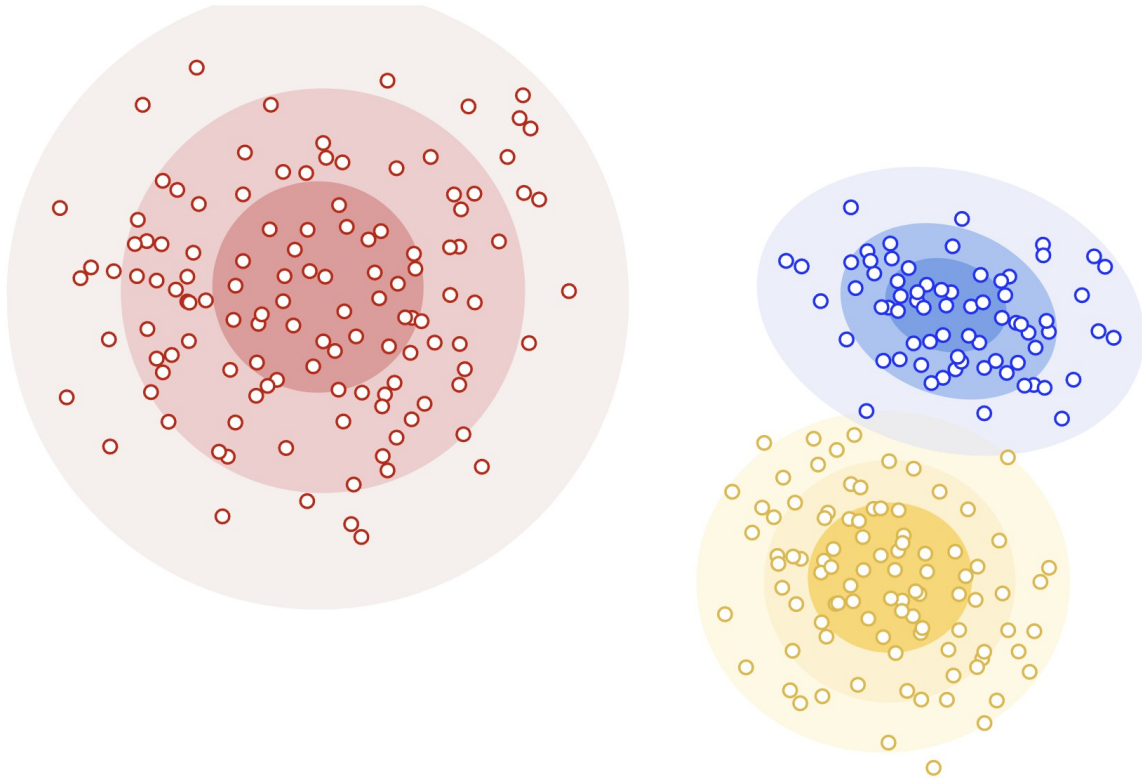
Quando un'area di un piano contiene molti punti dei dati vicini tra loro, ciò indica un'alta densità in quella regione. Questo suggerisce la possibile presenza di un cluster, poiché gli oggetti simili sono strettamente raggruppati.

Il clustering basato sulla densità collega le regioni di elevata densità degli esempi in cluster. Ciò consente distribuzioni di forma arbitraria, a patto che le aree dense possano essere collegate. Questi algoritmi hanno difficoltà con dati di densità variabile e in dimensioni elevate. Inoltre, per progettazione, questi algoritmi non assegnano gli outlier ai cluster. Un esempio di tecnica di clustering basata sulla densità è l'algoritmo **DBSCAN**.



Distribution-based Clustering

Questo approccio di clustering assume che i dati seguano una certa distribuzione, ad esempio la Gaussiana. L'algoritmo di clustering basato sulla distribuzione assegna gli oggetti a cluster in modo che seguano una distribuzione specifica. Un esempio di questo tipo di clustering è il **Gaussian Mixture Model (GMM)** clustering, che assume che i dati seguano una distribuzione gaussiana.

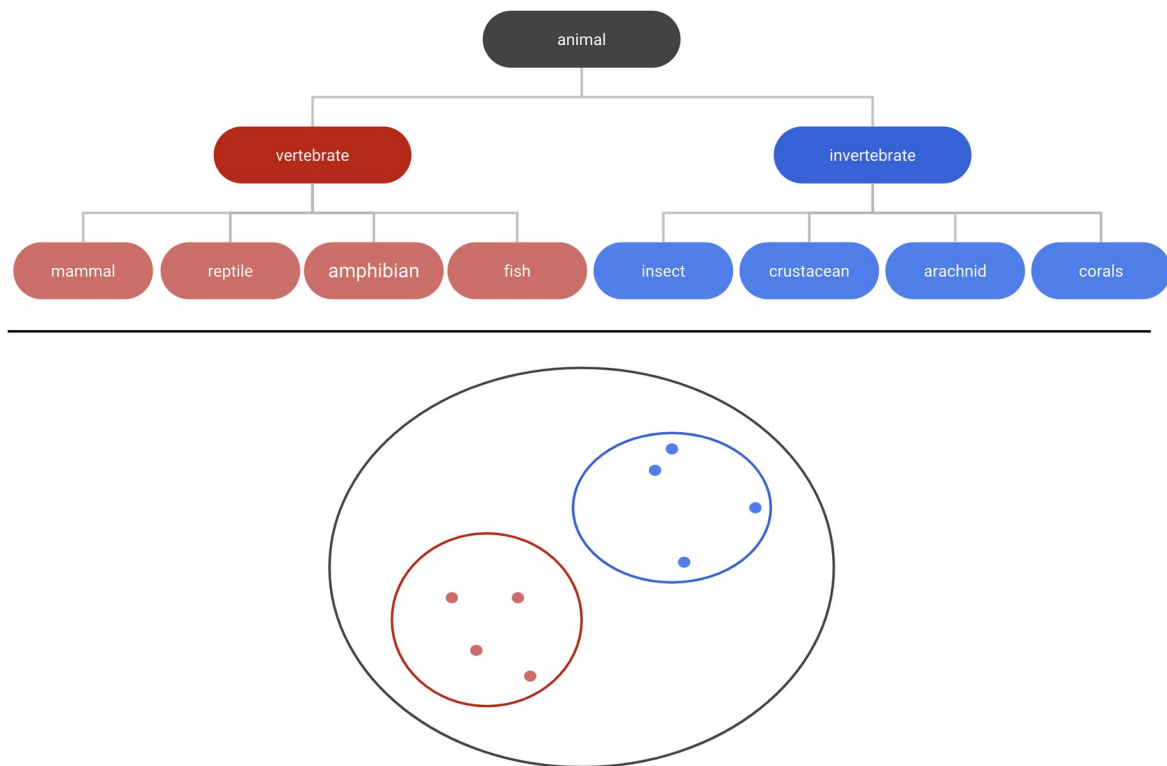


Se abbiamo delle distribuzioni gaussiane, all'aumentare della distanza dal centro della distribuzione, diminuisce la probabilità che un punto appartenga a quella distribuzione. Tuttavia, quando non si conosce il tipo di distribuzione nei dati, è opportuno utilizzare un algoritmo diverso. In figura le bande mostrano tale diminuzione di probabilità.

Hierarchical Clustering

L'approccio hierarchical clustering coinvolge la creazione di una gerarchia di cluster, in cui i cluster più piccoli sono combinati progressivamente per formare cluster più grandi. Questo processo può essere rappresentato graficamente come un dendrogramma, che mostra la struttura gerarchica dei cluster. Esistono diversi metodi per l'agglomerazione (combinazione) e la divisione dei cluster in questo approccio. Sklearn ad esempio fornisce

AgglomerativeClustering



Considerazioni finali

Questi quattro approcci forniscono diverse metodologie per suddividere i dati in cluster in base a criteri diversi, a seconda delle caratteristiche dei dati e degli obiettivi dell'analisi.

Quando si sceglie quale algoritmo da utilizzare è importante anche considerare se l'algoritmo scala nel dataset. In Machine Learning ci sono dataset con milioni di esempi ma non tutti gli algoritmi di clustering scalano efficientemente. Alcuni di loro calcolano la similarità fra tutte le coppie di esempi e questo vuol dire che a runtime il numero di esempi sale in $O(n^2)$. Tali algoritmi nella pratica quando si hanno milioni di esempi sono computazionalmente troppo costosi.

Ad esempio: **Kmeans** scala linearmente e ha una complessità di $O(n * k * i * d)$. Dove k sono i centroidi, i le iterazioni, d la dimensione dei dati.

DBASCN invece ha una complessità di $O(n * \log(n))$ ma in alcuni casi se i dati sono molto densi diventa $O(n^2)$, dove " n " rappresenta il numero di punti dati.

#Prendiamo i 3 dataset della volta scorsa ed effettuiamo il clustering dei punti.

```
from sklearn.model_selection import cross_val_predict, cross_val_score
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
import numpy as np
```

```

from sklearn import preprocessing
#importiamo 3 modelli da sklearn
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.mixture import GaussianMixture

#dataset generation
from sklearn.datasets import
make_blobs,make_moons,make_s_curve,make_circles
import matplotlib.pyplot as plt

n_samples = 1000
n_features = 2
n_classes = 2
seed = 42
noise_factor = 0.1
np.random.seed(seed)

# Genera il dataset con make_blobs
# X, y = make_blobs(n_samples=n_samples, n_features=n_features,
centers=n_classes, random_state=seed)
# X, y = make_blobs(n_samples=n_samples, n_features=n_features,
centers=n_classes, random_state=seed , cluster_std=[1.,2.] )
# X, y = make_moons(n_samples=n_samples, noise=noise_factor,
random_state=seed, )
X, y = make_circles(n_samples=n_samples, noise=noise_factor,
factor=0.1, random_state=seed)

## Data preprocessing
# Creiamo un dataset di esempio con due classi in proporzione diversa
scaler = preprocessing.MinMaxScaler() # range [0,1]
scaler.fit(X)
X = scaler.transform(X)

#mostro la distribuzione del dataset a video
plt.scatter( X[:,0], X[:,1], c=y, cmap="viridis")
plt.show()

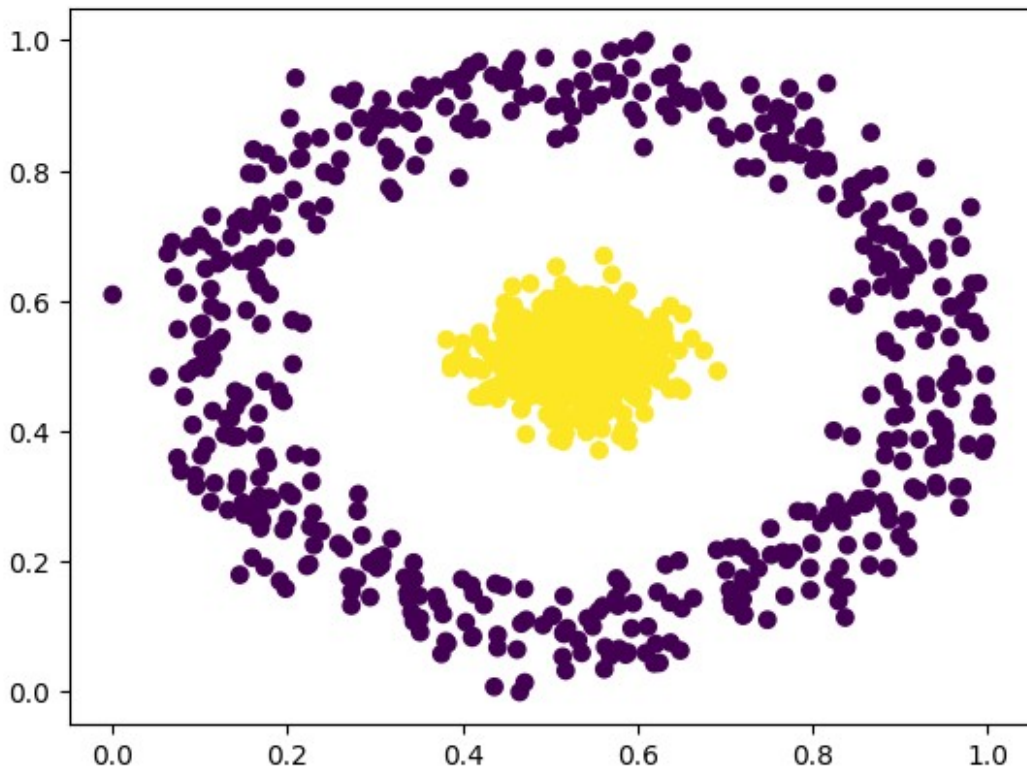
# Esegui uno splitting stratificato
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, stratify=y, random_state=seed) #stratify è importante

#DBSCAN e agglomerative clustering hanno un approccio "lazy",
#ovvero non richiedono un training ma sono algoritmi che possiamo
applicare direttamente ai dati di test.

def plot_clusters(x_axis,y_axis, y_predict, title):

```

```
plt.figure(figsize=(8, 6))
plt.scatter(x_axis, y_axis, c=y_predict, cmap='viridis')
plt.title(title)
plt.show()
```



Creazione e addestramento di K-Means

```
kmeans = KMeans(n_clusters=2, random_state=seed)
kmeans.fit(X_train)
```

```
kmeans_labels = kmeans.predict(X_test)
```

Creazione e addestramento di GMM

```
gmm = GaussianMixture(n_components=2, covariance_type='full',
random_state=seed)
gmm.fit(X_train)
gmm_labels = gmm.predict(X_test)
```

Creazione di DBSCAN

```
dbscan = DBSCAN(eps=0.2, min_samples=2 ) # i dati sono normalizzati in
[0,1] ed eps è il raggio che serve nell'intersezione
dbscan_labels = dbscan.fit_predict(X_test)
```

Creazione di AgglomerativeClustering

#il metodo "Ward" cerca di creare cluster che minimizzano la varianza.

```

#in questo modo si tengono i cluster omogenei e compatti.
ward = AgglomerativeClustering(n_clusters=2,linkage="ward" )#
ward=ward.fit(X_train)
ward_labels = ward.fit_predict(X_test)

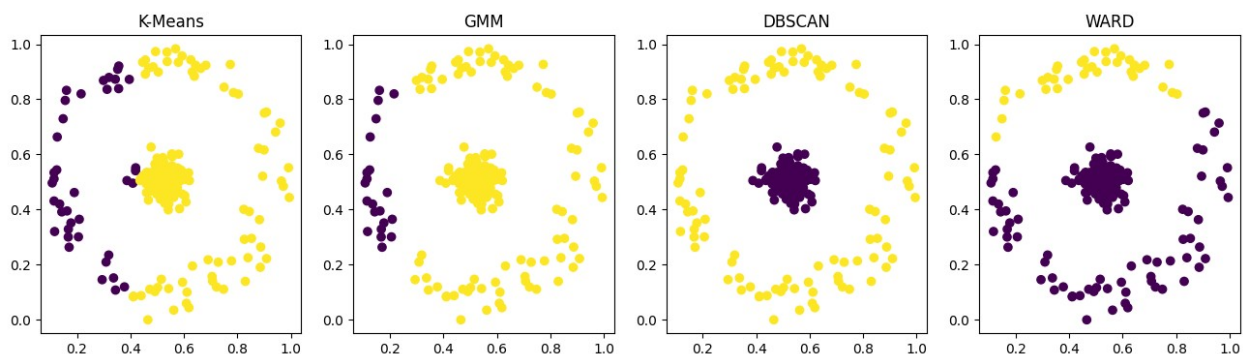
plt.figure(figsize=(16, 4))
# Plot K-Means
plt.subplot(1, 4, 1)
plt.scatter(X_test[:, 0], X_test[:, 1], c=kmeans_labels,
cmap='viridis')
plt.title('K-Means')

# Plot GMM
plt.subplot(1, 4, 2)
plt.scatter(X_test[:, 0], X_test[:, 1], c=gmm_labels, cmap='viridis')
plt.title('GMM')

# Plot DBSCAN
plt.subplot(1, 4, 3)
plt.scatter(X_test[:, 0], X_test[:, 1], c=dbscan_labels,
cmap='viridis')
plt.title('DBSCAN')

# Plot Agglomerative Clustering con metodo "Ward"
plt.subplot(1, 4, 4)
plt.scatter(X_test[:, 0], X_test[:, 1], c=ward_labels, cmap='viridis')
plt.title('WARD')
plt.show()

```



KMEANS, GMM e Hierarchical clustering, per loro definizione non possono funzionare in questo caso. Serve apportare modifiche alla distribuzione.

#Dimensionality Reduction

In campo di apprendimento automatico, la Principal Component Analysis (PCA) è una tecnica popolare per affrontare il problema della riduzione della dimensionalità.

##PCA

PCA trasforma un dataset ad alta dimensionalità in uno spazio a dimensioni inferiori, semplificandone la visualizzazione e accelerando altre analisi dei dati. Ma PCA non riduce semplicemente la dimensionalità dei dati: mira a catturare le strutture più significative nei dati, preservando il più possibile la variabilità.

Questo viene realizzato identificando le componenti principali in cui i dati variano di più e proiettando i dati su queste direzioni. Le componenti principali stesse sono combinazioni lineari delle caratteristiche originali, ortogonali tra loro, garantendo che non ci siano informazioni ridondanti. La prima componente principale cattura la varianza più alta nei dati, la seconda componente principale (ortogonale alla prima) cattura la seconda varianza più alta e così via. Rappresentando i dati in termini di queste componenti, PCA ci fornisce un modo per esprimere i dati in un numero ridotto di dimensioni che conserva le strutture essenziali.

Per fare un esempio, immagina un insieme di punti dati nello spazio tridimensionale che si trova principalmente su una superficie piatta. Nonostante i dati siano tridimensionali, la loro struttura intrinseca può essere catturata utilizzando solo due dimensioni: la superficie. PCA identifica questa superficie e ci consente di rappresentare ciascun punto dati tramite le sue coordinate sulla superficie anziché nello spazio tridimensionale completo.

PCA in pratica

Prendiamo il dataset di iris che avevamo visto. Aveva 4 features di input e ci veniva male mostrarle a video tutte insieme. Vogliamo ridurle per avere una visione migliore di queste features e scoprire eventuali patterns.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Carica il dataset Iris
iris = load_iris(as_frame=True)
X = iris.data # Carica i dati Iris

# Standardizzazione dei dati (mu 0, std 1)
X_std = StandardScaler().fit_transform(X)

print(f"Input shape: {X_std.shape}")
print(f"Numero di esempi: {X_std.shape[0]}")
print(f"Numero di features (Input Dimensionality): {X_std.shape[1]}")

Input shape: (150, 4)
Numero di esempi: 150
Numero di features (Input Dimensionality): 4

#Usiamo la pca per ridurre le features
```

```

n_comp = 2
pca = PCA(n_components=n_comp)

# Addestramento della PCA sui dati standardizzati
X_reduced = pca.fit_transform(X_std)

print(f"Dataset ridotto shape: {X_reduced.shape}")
print(f"Numero di esempi: {X_reduced.shape[0]}")
print(f"Numero di features (Input Dimensionality): {X_reduced.shape[1]}")

# Crea una figura 2D o 3D
fig = plt.figure()

if n_comp==3:
    ax = fig.add_subplot(111, projection='3d')
    ax.scatter(X_reduced[:, 0], X_reduced[:, 1], X_reduced[:, 2],
c=iris.target, cmap='viridis')
    ax.view_init(elev=10, azim=90) # Angolo di vista in gradi ed
elevazione
    ax.set_zlabel('Terza componente principale')
else:
    ax = fig.add_subplot(111)
    ax.scatter(X_reduced[:, 0], X_reduced[:, 1], c=iris.target,
cmap='viridis')

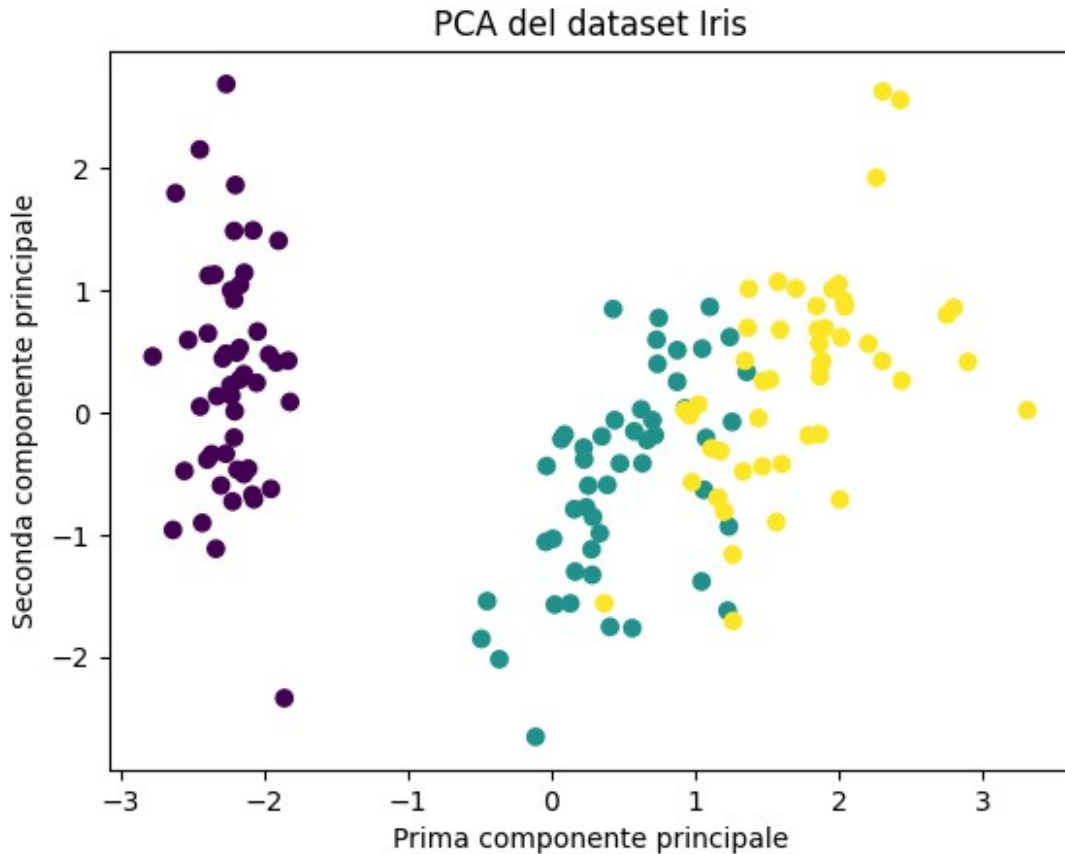
# Aggiungi etichette per gli assi
ax.set_xlabel('Prima componente principale')
ax.set_ylabel('Seconda componente principale')
ax.set_title('PCA del dataset Iris')
# Mostra il grafico
plt.show()

```

Dataset ridotto shape: (150, 2)

Numero di esempi: 150

Numero di features (Input Dimensionality): 2



Come si legge la PCA ?

1. Varianza spiegata da ciascun componente: Se ho 4 features, e li riduco a 2 la varianza spiegata è un grafo a 2 barre dove ogni barra indica quanto una componente riesce a rappresentare gli esempi rappresentati dalle 4 features. Es nel nostro caso la prima componente cattura il 70% della varianza totale dei dati, la seconda cattura il 30%. La somma delle varianze spiegate da tutte le componenti principali sarà uguale a 1.
2. Varianza spiegata cumulativa: Se ordiniamo le componenti e li sommiamo, ad ogni somma abbiamo la varianza spiegata dalle prime n componenti rispetto alla varianza totale. E quindi ti fornisce una visione collettiva.

Sono metriche utili per decidere il numero di componenti.

```
# Varianza spiegata
explained_variance = pca.explained_variance_ratio_
explained_variance_cumulative = np.cumsum(explained_variance)

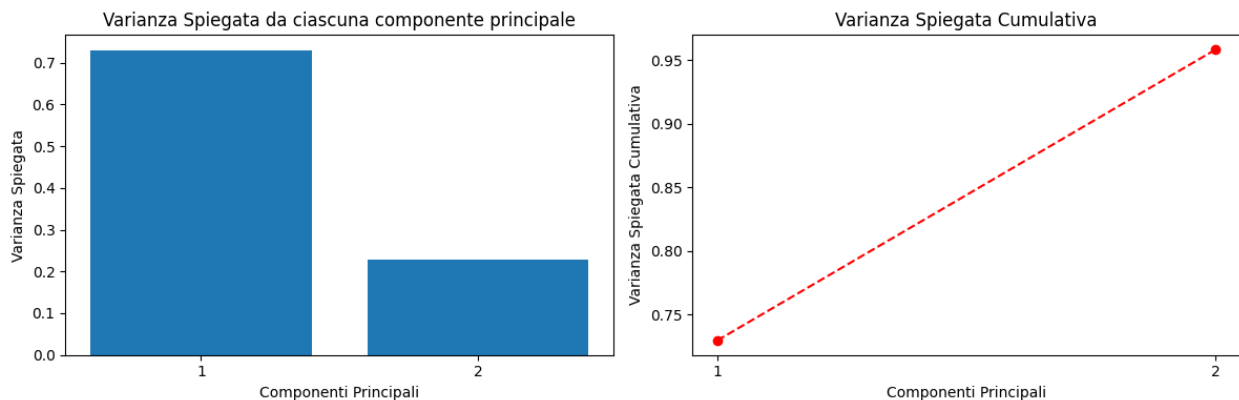
# Plot della varianza spiegata
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.bar(range(1, len(explained_variance) + 1), explained_variance)
```

```
plt.xticks(range(1, len(explained_variance) + 1))

plt.xlabel('Componenti Principali')
plt.ylabel('Varianza Spiegata')
plt.title('Varianza Spiegata da ciascuna componente principale')

# Plot della varianza spiegata cumulativa
plt.subplot(1, 2, 2)
plt.plot(range(1, len(explained_variance_cumulative) + 1),
         explained_variance_cumulative, marker='o', linestyle='--', color='r')
plt.xticks(range(1, len(explained_variance) + 1))
plt.xlabel('Componenti Principali')
plt.ylabel('Varianza Spiegata Cumulativa')
plt.title('Varianza Spiegata Cumulativa')
plt.tight_layout()

plt.show()
```



Nella riduzione di dimensionalità, per le tecniche quali PCA o altre ad esempio AutoEncoders, alcune delle informazioni originali vengono eliminate. Le componenti principali estratte rappresentano una sintesi delle informazioni contenute nelle features originali. Esiste un Trade-Off tra dimensione e informazione.

###Approfondimento PCA

```
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
data = load_iris()
X = data.data
y = data.target
y_std = y
# Standardize the data
X_std = StandardScaler().fit_transform(X)
```

```

#Calcola la matrice delle covarianze
covariance_matrix = np.cov(X_std.T)

#computa EigenVector ed EigenValues
eigenvalues, eigenvectors = np.linalg.eig(covariance_matrix)

#Sort by descending eigenvalues.
#This will help in deciding which eigenvector(s) can be dropped
without losing significant information.

# Create pairs (eigenvalue, eigenvector)
eig_pairs = [(np.abs(eigenvalues[i]), eigenvectors[:, i]) for i in
range(len(eigenvalues))]

# Sort the pairs based on the eigenvalues
eig_pairs.sort(key=lambda x: x[0], reverse=True)

# Scegli top-k eigenvalues
num_components = 2
matrix_w = np.hstack([eig_pairs[i][1].reshape(X_std.shape[1], 1) for i
in range(num_components)])

X_pca = X_std.dot(matrix_w)

print(f"Prima avevamo una shape sull'input di {X_std.shape}")
print(f"Ora la nuova shape è {X_pca.shape}")

Prima avevamo una shape sull'input di (150, 4)
Ora la nuova shape è (150, 2)

```

1. **Calcola la Matrice delle Covarianze:** essa rappresenta le relazioni tra le variabili nel dataset.
2. **Calcola Autovalori ed Autovettori:** Questi rappresentano le caratteristiche principali della variazione nei dati. Gli autovettori rappresentano una direzione nello spazio multidimensionale delle variabili originali, mentre gli autovalori indicano quanto le dimensioni in queste direzioni vengano scalate o compressi
3. **Ordina Autovalori ed Autovettori in modo decrescente:** Questo passo è cruciale perché ti permette di identificare i componenti più importanti in base alla varianza spiegata
4. **Seleziona i Top-k Componenti:** Puoi scegliere il numero desiderato di componenti principali (autovettori) da mantenere per la riduzione della dimensionalità. Questi rappresentano le dimensioni più importanti nei dati.
5. **Proietta i Dati nel Nuovo Spazio delle Feature:** Utilizzando gli autovettori selezionati, proietta i dati originali nello spazio delle feature ridotto attraverso una trasformazione lineare, solitamente calcolata come un prodotto scalare tra i dati e gli autovettori.

Anomaly Detection

Il problema trova diverse applicazioni ad esempio rilevazione di Spam, transazioni finanziarie fraudolente, controllo della qualità industriale. In ciascuno di questi contesti, l'obiettivo è identificare eventi o osservazioni inusuali che richiedono attenzione o azioni speciali.

Caratteristiche delle Anomalie

Le anomalie possono assumere diverse forme. Possono essere dei dati isolati che si discostano dai punti dati circostanti, sequenze di dati insolite, o cluster di dati che rappresentano comportamenti insoliti. La definizione di "anomalia" dipende dal contesto e dalle specifiche dell'applicazione.

Techniche di anomaly detection Alcuni dei metodi più comuni includono la statistica descrittiva, i metodi basati su soglie, i modelli di machine learning (come Isolation Forest, One-Class SVM, Autoencoder), e l'apprendimento automatico supervisionato quando si dispone di etichette per gli outlier. Vedremo alcune tecniche unsupervised per il rilevamento di anomalie.

La valutazione dei modelli di Anomaly Detection coinvolge l'uso di metriche come la precisione, il richiamo e l'F1-score. Poiché le anomalie sono spesso rare, è importante considerare l'equilibrio tra falsi positivi e falsi negativi.

```
import time
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import make_blobs, make_moons

# Example settings
n_samples = 300
outliers_fraction = 0.15
n_outliers = int(outliers_fraction * n_samples)
n_inliers = n_samples - n_outliers
rng = np.random.RandomState(42)

matplotlib.rcParams["contour.negative_linestyle"] = "solid"

# Define datasets
blobs_params = dict(random_state=0, n_samples=n_inliers, n_features=2)

dataset_1 = make_blobs(centers=[[0, 0], [0, 0]], cluster_std=0.5,
**blobs_params)[0]
dataset_2 = make_blobs(centers=[[2, 2], [-2, -2]], cluster_std=[0.5,
0.5], **blobs_params)[0]
dataset_3 = make_blobs(centers=[[2, 2], [-2, -2]], cluster_std=[1.5,
0.3], **blobs_params)[0]
dataset_4 = 4.0*( make_moons(n_samples=n_samples, noise=0.05,
random_state=0)[0] - np.array([0.5, 0.25]) )

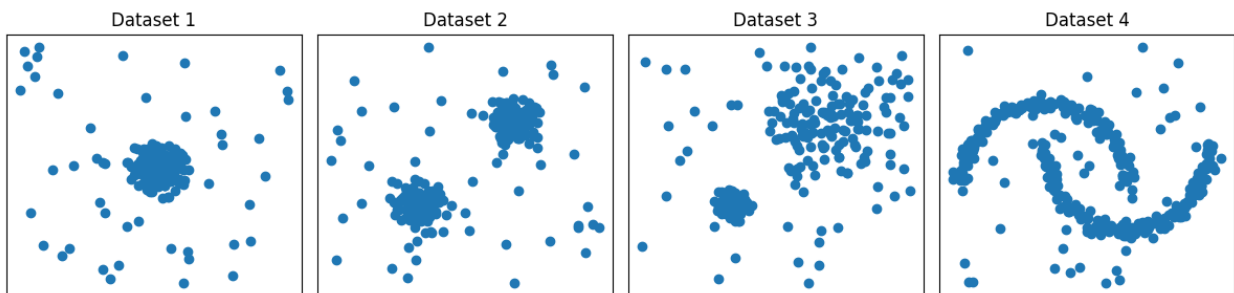
datasets = [dataset_1, dataset_2, dataset_3, dataset_4]
```

```

plt.figure(figsize=(15, 3))
for i, X in enumerate(datasets):
    #add outliers
    X = np.concatenate([X, rng.uniform(low=-6, high=6,
size=(n_outliers, 2))], axis=0)
    # 0 = rng.uniform(low=-6, high=6, size=(n_outliers, 2))
    plt.subplot(1, 5, i + 1)
    plt.scatter(X[:, 0], X[:, 1] )
    # plt.scatter(0[:, 0], 0[:, 1] )
    plt.xticks(())
    plt.yticks(())
    plt.title(f'Dataset {i + 1}')

plt.tight_layout()
plt.show()

```



```

import time
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import make_blobs, make_moons

# Example settings
n_samples = 300
outliers_fraction = 0.15
n_outliers = int(outliers_fraction * n_samples)
n_inliers = n_samples - n_outliers
rng = np.random.RandomState(42)

matplotlib.rcParams["contour.negative_linestyle"] = "solid"

# Define datasets
blobs_params = dict(random_state=0, n_samples=n_inliers, n_features=2)

dataset_1 = make_blobs(centers=[[0, 0], [0, 0]], cluster_std=0.5,
**blobs_params)[0]
dataset_2 = make_blobs(centers=[[2, 2], [-2, -2]], cluster_std=[0.5,

```

```

0.5], **blobs_params)[0]
dataset_3 = make_blobs(centers=[[2, 2], [-2, -2]], cluster_std=[1.5,
0.3], **blobs_params)[0]
dataset_4 = 4.0*( make_moons(n_samples=n_samples, noise=0.05,
random_state=0)[0] - np.array([0.5, 0.25]) )

datasets = [dataset_1, dataset_2, dataset_3, dataset_4]

from sklearn.ensemble import IsolationForest
from sklearn.svm import OneClassSVM
from sklearn.neighbors import LocalOutlierFactor
from sklearn.covariance import EllipticEnvelope

#Ho un punto e voglio classificarlo come simile o non simile ai suoi
vicini
#Costruisco degli alberi decisionali per farlo e vedo quanto sono
profondi
#le anomalie hanno alberi poco profondi perchè è diverso dai suoi
vicini.
isolation_forest = IsolationForest(contamination=outliers_fraction)

# One-Class SVM (Support Vector Machine)
oc_svm = OneClassSVM(nu=outliers_fraction, kernel="rbf", gamma=0.1)
# Imposta il parametro nu, che controlla la percentuale di punti
normali

lof = LocalOutlierFactor(n_neighbors=35,
contamination=outliers_fraction, novelty=True ) # Imposta
novelty=True per il rilevamento delle anomalie
# LOF è un valore calcolato sulla base della densità locale di un
punto dato, e misura quanto un punto è significativamente diverso (e
quindi è un outlier) rispetto ai suoi vicini.

# Elliptic Envelope presuppone che i dati abbiano distribuzione
gaussiana e guarda alla loro covarianza (crea solo delle ellissi)
elliptic = EllipticEnvelope(contamination=outliers_fraction,
random_state=42) # Imposta la percentuale di contaminazione

anomaly_algorithms=[
    ("Robust covariance",elliptic ),
    ("Isolation Forest",isolation_forest ),
    ("One-Class SVM",oc_svm ),
    ("Local Outlier Factor",lof )
]

xx, yy = np.meshgrid(np.linspace(-7, 7, 150), np.linspace(-7, 7, 150))

plt.figure(figsize=(len(anomaly_algorithms) * 4, len(datasets) * 4))
plt.subplots_adjust(
    left=0.02, right=0.98, bottom=0.01, top=0.96, wspace=0.05,

```



```

hspace=0.2
)

plot_num = 1
rng = np.random.RandomState(42)

for i_dataset, X in enumerate(datasets):
    # Add outliers
    X = np.concatenate([X, rng.uniform(low=-6, high=6,
size=(n_outliers, 2))], axis=0)

    for name, algorithm in anomaly_algorithms:
        t0 = time.time()
        if name == "Local Outlier Factor" and algorithm.novelty:
            y_pred = algorithm.fit(X).predict(X)
        else:
            y_pred = algorithm.fit_predict(X)

        t1 = time.time()
        plt.subplot(len(datasets), len(anomaly_algorithms), plot_num)
        plt.title(name, size=18)

        Z = -algorithm.decision_function(np.c_[xx.ravel(),
yy.ravel()]).reshape(xx.shape)
        plt.contourf(xx, yy, Z, cmap=plt.cm.Blues, alpha=0.6)

        colors = np.array(["#377eb8", "#ff7f00"])
        plt.scatter(X[:, 0], X[:, 1], s=10, color=colors[(y_pred + 1)
// 2])

        plt.xlim(-7, 7)
        plt.ylim(-7, 7)
        plt.xticks(())
        plt.yticks(())
        plt.text(
            0.99,
            0.01,
            ("%.2fs" % (t1 - t0)).lstrip("0"),
            transform=plt.gca().transAxes,
            size=15,
            horizontalalignment="right",
        )
        plot_num += 1

plt.show()

```

